

```

from typing import List, Dict, Set
from collections import defaultdict
from functools import reduce

# 1. Total screen time per user
def total_time_per_user(logs: List[Dict]) -> Dict[str, float]:

    users = {log["user"] for log in logs}

    return {
        user: reduce(
            lambda total, log: total + (log["duration"] if log["user"] == user
else 0),
            logs,
            0.0
        )
        for user in users
    }

def most_active_users(logs: List[Dict], k: int) -> List[str]:

    time_per_user = defaultdict(float)

    for log in logs:
        time_per_user[log["user"]] += log["duration"]

    sorted_users = sorted(
        time_per_user.items(),
        key=lambda x: x[1],
        reverse=True
    )

    return [user for user, _ in sorted_users[:k]]

def unique_actions(logs: List[Dict]) -> Set[str]:

    return {log["action"] for log in logs}

if __name__ == "__main__":

```

```
logs = [  
    {"user": "220101", "action": "YouTube", "duration": 30.5},  
    {"user": "220102", "action": "Instagram", "duration": 20},  
    {"user": "220101", "action": "WhatsApp", "duration": 15},  
    {"user": "220103", "action": "YouTube", "duration": 40},  
    {"user": "220102", "action": "Chrome", "duration": 10}  
]  
  
print("Total Time Per User:")  
print(total_time_per_user(logs))  
  
print("\nMost Active Users:")  
print(most_active_users(logs, 2))  
  
print("\nUnique Actions:")  
print(unique_actions(logs))
```

OUTPUT

Total Time Per User:

{'220102': 30.0, '220101': 45.5, '220103': 40.0}

Most Active Users:

['220101', '220103']

Unique Actions:

{'WhatsApp', 'YouTube', 'Chrome', 'Instagram'}

Complexity Analysis

a. Time Complexity for Computing Top K Users

Let n be the number of log records.

Aggregating total time per user requires iterating through all logs $\rightarrow O(n)$.

Sorting the users based on total duration requires $O(u \log u)$ where u is the number of unique users.

Since

$u \leq n$, the overall complexity becomes:

$$O(n + u \log u)$$

In the worst case:

$$O(n \log n)$$

b. Space Complexity of Storing Intermediate Results

A dictionary is used to store the total activity time per user.

If u is the number of unique users, the dictionary requires $O(u)$ space.

Additionally, the set used to store unique actions requires $O(a)$ space where a is the number of unique actions.

Therefore, the overall space complexity is:

$O(u+a)$